Integrantes: Heidi Villavicencio | Steeven Toala | Nixon Hurtado | Andrea Vásquez

1. TÍTULO

Denominación del Entregable: Configuración de Infraestructura como Código (IaC), Contenedores y Docker Compose (EDT 2.6)

2. INTRODUCCIÓN

Para garantizar reproducibilidad en los entornos de desarrollo, pruebas y producción sin incurrir en costos de licencias comerciales ni saturar los recursos de hardware, el sistema **SIGE** implementa una arquitectura eficiente y desacoplada mediante **Docker Compose**: servicios locales contenerizados (**PostgreSQL 18**, MinIO, LanguageTool, **Backend Node.js + Express v5**, **Frontend React + Vite**) e inferencia semántica delegada a la **API de Google Gemini** a través de HTTPS sin costo de GPU local.

3. OBJETIVO

3.1. Objetivo General

Diseñar, configurar y orquestar la arquitectura de contenedores Docker locales para el ecosistema completo de SIGE, asegurando reproducibilidad y despliegue automatizado en la FISEI UTA.

3.2. Objetivos Específicos

1. Configurar los 6 servicios del `docker-compose.yml` (PostgreSQL 18, MinIO, MinIO-Init, LanguageTool, Backend Node.js Express v5 y Frontend React + Vite).
2. Establecer las redes virtuales internas y políticas de persistencia en volúmenes nombrados.
3. Parametrizar las variables de entorno y definir los endpoints de Health Check.

4. ALCANCE

4.1. Límites del Despliegue (In-Scope)

- Orquestación local para desarrollo y producción On-Premise en un host con Docker Engine 24+.

4.2. Exclusiones (Out-of-Scope)

- Clústeres Kubernetes distribuidos en múltiples nodos de nubes comerciales de pago.

5. DEFINICIONES

- **Docker Compose:** Herramienta para definir y ejecutar aplicaciones multi-contenedor mediante archivos YAML.
- **Named Volume:** Mecanismo de persistencia de datos gestionado por Docker independiente del ciclo de vida del contenedor.
- **Healthcheck:** Instrucción que comprueba periódicamente el estado operativo interno de un servicio.

6. RESPONSABILIDADES

Integrante	Rol en el Entregable	Responsabilidad Específica
Steeven Toala	Líder / Arquitecto	Diseño de Dockerfiles, archivo `docker-compose.yml` y configuración de redes.
Nixon Hurtado	Backend / DBA	Configuración del contenedor de PostgreSQL 18 y persistencia de volúmenes.
Ing. Paulo Torres	Docente Tutor	Validación y prueba del despliegue en un único comando.

Tabla 1: Matriz de Responsabilidades del Entregable EDT 2.6.

7. NORMATIVA LEGAL Y TÉCNICA

- **Open Container Initiative (OCI) Image Specification v1.1.**
- **CIS Docker Benchmark v1.6.0:** Guía de endurecimiento de seguridad en contenedores.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

La topología de servicios contenerizados conectados bajo la red puente `sige-network` se estructura de la siguiente manera:

- **Servicios Locales Contenerizados:** Base de datos relacional (**PostgreSQL 18**), almacenamiento de objetos S3 (**MinIO**), corrector ortográfico/gramatical local de alta velocidad (**LanguageTool**), backend REST (**Node.js + Express v5**) y cliente web (**React + Vite + Nginx**).
- **Asistencia Semántica de Inteligencia Artificial:** La inferencia semántica para redacción institucional se delega a la **API de Google Gemini** a través de peticiones seguras HTTPS (REST API), optimizando la infraestructura local a cero sobrecosto de hardware.

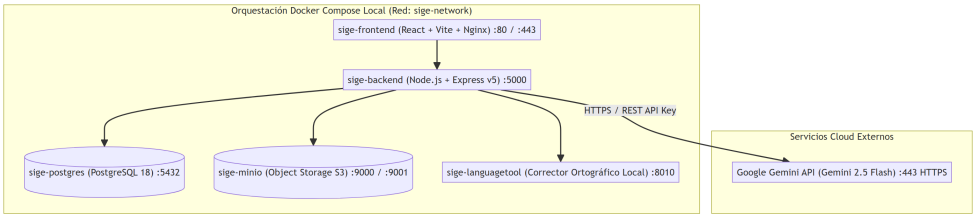


Figura 1: Arquitectura de Contenedores y Mapeo de Puertos en Docker Compose.

9. DESCRIPCIÓN DE ACTIVIDADES

9.1. Especificación del Archivo `docker-compose.yml`

```
version: '3.8'

services:
  # 1. Base de Datos Relacional PostgreSQL 18
  sige-postgres:
    image: postgres:18-alpine
    container_name: sige-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: sige_db
      POSTGRES_USER: sige_admin
      POSTGRES_PASSWORD: SigeSecurePassword2026!
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - sige-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U sige_admin -d sige_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  # 2. Servidor de Objetos MinIO (Almacenamiento Seguro S3)
  sige-minio:
    image: minio/minio:RELEASE.2024-05-10T01-41-38Z
    container_name: sige-minio
    restart: unless-stopped
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: sige_minio_admin
      MINIO_ROOT_PASSWORD: SigeMinioSecretKey2026!
    ports:
      - "9000:9000"   # API S3 Endpoint
      - "9001:9001"   # Consola Web de Administracion
    volumes:
      - minio_data:/data
    networks:
      - sige-network

  # 3. Inicializador de Bucket MinIO (Crea el bucket de evidencias automaticamente)
```

```
  sige-minio-init:
    image: minio/mc:latest
    container_name: sige-minio-init
    depends_on:
      - sige-minio
    entrypoint: >
      /bin/sh -c "
      until (/usr/bin/mc config host add sigeMinio http://sige-minio:9000
          sige_minio_admin SigeMinioSecretKey2026!) do echo 'Esperando a MinIO...' &&
          sleep 2; done;
      /usr/bin/mc mb sigeMinio/sige-evidencias-bucket --ignore-existing;
      /usr/bin/mc anonymous set download sigeMinio/sige-evidencias-bucket;
      exit 0;
      "
    networks:
      - sige-network

  # 4. Servidor Local LanguageTool (Corrector Ortografico en Espanol)
  sige-languagetool:
    image: erikvl87/languagetool:latest
    container_name: sige-languagetool
    restart: unless-stopped
    environment:
      langtool_languageModel: "/ngrams"
      Java_Xms: "512m"
      Java_Xmx: "1g"
    ports:
      - "8010:8010"
    networks:
      - sige-network

  # 5. Backend REST API (Node.js + Express v5)
  sige-backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: sige-backend
    restart: unless-stopped
    depends_on:
      sige-postgres:
        condition: service_healthy
      sige-minio:
        condition: service_started
      sige-languagetool:
        condition: service_started
    environment:
      - NODE_ENV=development
      - PORT=5000
      - DB_HOST=sige-postgres
      - DB_PORT=5432
      - DB_NAME=sige_db
      - DB_USER=sige_admin
      - DB_PASSWORD=SigeSecurePassword2026!
      - MINIO_ENDPOINT=sige-minio
      - MINIO_PORT=9000
      - MINIO_ACCESS_KEY=sige_minio_admin
      - MINIO_SECRET_KEY=SigeMinioSecretKey2026!
      - MINIO_BUCKET=sige-evidencias-bucket
```

```
      - LANGUAGETOOL_URL=http://sige-languagetool:8010
      - GEMINI_API_KEY=${GEMINI_API_KEY}
      - JWT_SECRET=SigeSuperSecretKeyForTokenSigning2026UTAFISEISoftwareEngineering!
      - JWT_EXPIRES_IN=30m
    ports:
      - "5000:5000"
    networks:
      - sige-network

  # 6. Frontend React + Vite (Servido con Nginx)
  sige-frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: sige-frontend
    restart: unless-stopped
    depends_on:
      - sige-backend
    ports:
      - "80:80"
    networks:
      - sige-network

networks:
  sige-network:
    driver: bridge

volumes:
  postgres_data:
    driver: local
  minio_data:
    driver: local
```

9.2. Comandos de Despliegue y Verificación Local

9.3. Inicio de Servicios

```
# 1. Clonar el repositorio y posicionarse en la raiz
cd d:/2Proyectos/GESTION_DE_PROYECTOS_DE_SOFTWARE/FISEI_Project

# 2. Levantar todos los contenedores en segundo plano
docker compose up -d

# 3. Verificar estado operativo de los servicios
docker compose ps
```

9.4. Endpoints de Verificación de Salud (Health Checks)

- **Frontend Web:** `http://localhost:80`
- **Backend REST API:** `http://localhost:5000/api/health`
- **Consola MinIO:** `http://localhost:9001` (User: `sige_minio_admin` / Pass: `SigeMinioSecretKey2026`
- **API LanguageTool:** `http://localhost:8010/v2/check?text=Hola%20mundo&language=es`

- **Asistente Gemini AI:** Conectividad HTTPS directa con `https://generativelanguage.googleapis.co` (verificable vía `/api/ia/status` en backend con API Key institucional).

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Requisitos / UI-UX		04/09/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		04/09/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Configuración de docker-compose.yml con los 6 servicios y comandos de verificación / Pendiente de revisión	Steeven Toala	**Pendiente de Revisión**

12. ANEXOS

Anexo 1: Script de Inicialización de Redes y Volúmenes de Docker

Comandos `docker volume inspect` y políticas de reinicio automático ante fallos de contenedor.